Video Activity Recognition

Using 3D CNN & Grad-CAM Explainable AI

Spatiotemporal Feature Learning for Human Action Classification paired with Visual Attention Heatmaps.

Department: AI & Machine Learning (AIML)	**Core Architecture: 3D CNN (Conv3D)**
Framework: PyTorch & OpenCV	**Explainability: 3D Grad-CAM Heatmaps**
Dataset: UCF101 Action Categories	**Target Environment: Local CPU Execution**

Problem Statement & Motivation

Why traditional 2D CNNs fail and why 3D CNNs are required:

Limitations of Standard 2D CNNs

- Evaluates static frames independently (Spatial H × W).
- Extracts posture only ('What is in the image').
- Ignores temporal motion trajectories across time.
- Cannot distinguish opposite actions with identical poses.

Advantages of 3D CNNs (AIML Solution)

- Applies 3D kernels across Spatial + Temporal (H × W × Time).
- Captures posture AND motion trajectory simultaneously.
- Preserves time-series continuity across frames.
- Accurately classifies sequential human actions.

Key Concept: 3D Convolutions are required because motion across consecutive video frames carries the semantic meaning of human activities.

System Pipeline Architecture

Complete data flow from video input to model prediction and heatmaps:

1. Input Video	2. Preprocessing	3. 3D CNN	4. Classifier	5. Grad-CAM
Raw video clip (MP4 / AVI format)	16 uniform frames Resized 112×112 RGB	Spatiotemporal Conv Feature maps	Softmax Layer Action class label	3D Heatmap overlay Visual attention map

Tensor Pipeline Flow: Input Video → OpenCV Frame Extractor → Tensor (Batch, 3, 16, 112, 112) → 3D CNN Forward Pass → Softmax Class Logits → Grad-CAM Backprop → Visual Overlay.

3D CNN Model Architecture & Tensor Shapes

PyTorch Simple3DCNN network specifications:

Network Layer	Kernel Parameters	Output Tensor Dimension
Input Video Clip	16 RGB Frames @ 112×112 resolution	`(B, 3, 16, 112, 112)`
Conv3D Layer 1	3 → 64 filters, kernel (3,3,3), padding=1	`(B, 64, 16, 112, 112)`
MaxPool3D 1	kernel=(1,2,2), stride=(1,2,2)	`(B, 64, 16, 56, 56)`
Conv3D Layer 2 (Target)	64 → 128 filters, kernel (3,3,3), padding=1	`(B, 128, 16, 56, 56)`
MaxPool3D 2	kernel=(2,2,2), stride=(2,2,2)	`(B, 128, 8, 28, 28)`
Linear FC1	Linear(128 × 8 × 28 × 28, 256)	`(B, 256)`
Linear FC2 (Classifier)	Linear(256, num_classes)	`(B, num_classes)`

Video Preprocessing & Clip Generation

Standardizing raw video clips for robust tensor creation:

1. Uniform Frame Sampling	2. Resizing & Color Space	3. Normalization
Reads frames using OpenCV VideoCapture. Computes uniform indices with np.linspace to extract exactly 16 frames per clip.	Resizes every frame to standard 112×112 resolution. Converts BGR to RGB color space for PyTorch compatibility.	Scales uint8 pixel values [0, 255] to float32 [0, 1]. Applies standardization for stable gradient flow.

Dataset Folder Structure:
dataset/<class_name>/<video_file>.mp4 (e.g., dataset/Walking/v_Walking_g01_c01.avi)

Explainable AI (XAI) using 3D Grad-CAM

Visualizing where the model focuses when predicting actions:

Step	Grad-CAM Operation	Technical Explanation
1. Target Layer	Select conv2 final 3D conv layer	Hooks into high-level spatiotemporal feature maps.
2. Gradients	Compute $\partial y^c / \partial A^k$	Calculates backprop gradients of target class score.
3. Weighting	Global Average Pooling of gradients	Obtains importance weights α_k^c per feature map.
4. Heatmap	Apply ReLU($\sum \alpha_k^c A^k$)	Isolates positive contributing spatial-temporal regions.
5. Visualization	OpenCV COLORMAP_JET superimposition	Overlays heatmap onto original video frames for output.

Training Strategy & Hyperparameters

Model training configuration and local CPU optimization:

Hyperparameter	Value	Description / Rationale
Loss Function	CrossEntropyLoss	Multi-class categorical cross-entropy loss function.
Optimizer	Adam (lr = 0.001)	Adaptive gradient optimizer for fast convergence.
Batch Size	4 / 8 clips	Prevents RAM overflow during local CPU execution.
Sequence Length	16 frames	Optimal temporal window size for video clip modeling.

CPU Optimization Strategy:

- Trained on modular dataset subset (2-3 distinct activity classes) for fast execution on ThinkPad i7.
- Model checkpointing automatically saves highest accuracy weights to models/best_3dcnn.pth.

Performance Evaluation Framework

Quantitative performance metrics for model validation:

Metric	Formula	Evaluation Purpose
Accuracy	`(TP + TN) / Total`	Overall proportion of correctly classified video clips.
Precision	`TP / (TP + FP)`	Measures exactness (low false positive action triggers).
Recall	`TP / (TP + FN)`	Measures sensitivity in capturing true action occurrences.
F1-Score	`2 × (P × R) / (P + R)`	Harmonic mean evaluating class-imbalance performance.

Evaluated using sklearn.metrics.classification_report along with multi-class confusion matrix heatmaps to verify class prediction quality.

Final Output Deliverable & Demo

3-part visualization output for project presentation:

1. Input Video Stream	2. Prediction & Confidence	3. Grad-CAM Heatmap
Raw video clip sampled into 16 RGB frame sequences. *Example: Walking video clip*	Model predicts target action class with confidence score. **Output:** 'Walking' (98.4% Confidence)	Superimposes red/yellow attention heatmaps on frames. **Verification:** Confirms focus on moving legs.

Conclusion & Future Scope

Key Project Accomplishments

• Built complete end-to-end 3D CNN pipeline for video action recognition.

• Successfully integrated 3D Grad-CAM for visual decision interpretability.

• Optimized codebase for local CPU execution.

• Proved spatiotemporal feature learning over 2D frame baselines.

Future Enhancements

• Upgrade backbone to **Video Swin Transformers / SlowFast Networks**.

• Deploy model to edge hardware (Raspberry Pi / NVIDIA Jetson).

• Multi-modal fusion combining audio signals with 3D video features.

• Expand real-time video surveillance action detection.

Thank You!

Department of Artificial Intelligence & Machine Learning (AI & ML)